16-11-2025

Agenda:

Clustering Evaluation:

→ Elbow method (completed) → (kmeans) wcss → optimal value of k

→ Accuracy → optimal value of k (k-nearest)

→ Silhouette Coefficient

→ Calinski-Harabasz

→ project (ML)

→ Silhouette Coefficient

→ Measure how well each point belongs to its cluster

→ check two thing here:

(1) How close a point is to its own cluster.

(2) How far it is from the nearest other cluster.

point	x	y	cluster
A	1	1	C1
B	2	1	C1
C	1	2	C1
D	8	8	C2
E	9	8	C2

C1 → A, B, C

C2 → D, E

Step 1 : ① Compute silhouette for point A (1, 1)

→ compute $a(A)$ (average distance to point in its own cluster)

$$\text{Dist}(A,B) \rightarrow \sqrt{(2-1)^2 + (1-1)^2} = \sqrt{1} = 1$$
$$\text{Dist}(A,C) \rightarrow \sqrt{(1-1)^2 + (2-1)^2} = \sqrt{1} = 1$$

$$a(A) = (1+1)/2 = 1.0$$

② compute $b(A)$ (avg distance to point in nearest other cluster (2))

$$\text{Dist}(A,D) = \sqrt{(8-1)^2 + (8-1)^2} = \sqrt{49+49} = \sqrt{98} \approx 9.899$$
$$\text{Dist}(A,E) = \sqrt{(9-1)^2 + (8-1)^2} = \sqrt{64+49} = \sqrt{113} \approx 10.630$$

Average:

$$b(A) = (9.899 + 10.630)/2 \approx 10.265$$

③ Silhouette $s(A)$

$$s(A) = \frac{b-a}{\max(a,b)} = \frac{10.265 - 1}{10.265} \approx 0.903$$

Step 2 : $s(B) = 2.1$

① dist B, A
dist B, C → average → $a(B)$

② dist B, D
dist B, E → average → $b(B)$

③ $s(B) \rightarrow \frac{b-a}{\max(a,b)} \rightarrow \approx 0.873$

Step 3: $s(C) = (1,2)$

$\rightarrow s(C) \rightarrow \approx 0.879$

Step 4: $s(D) = (8,8)$

$s(D) \approx 0.894$

Step 5: $s(E) = (9,8)$

$s(E) \approx 0.902$

Final step: Final Silhouette score (average)

$$S = \frac{s(A) + s(B) + s(C) + s(D) + s(E)}{5}$$

$$\approx 0.889$$

Interpretation:

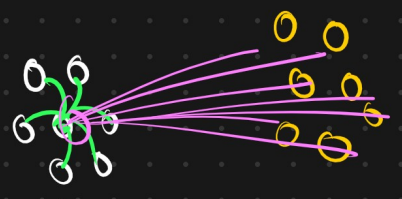
0.7 to 1.0 — perfect clustering or clear separation

0.5 to 0.7 — Good, reasonable clusters

0.25 to 0.5 → weak clusters

< 0.25 — No cluster (random blobs)

Negative → Misclustered (belong to another cluster)

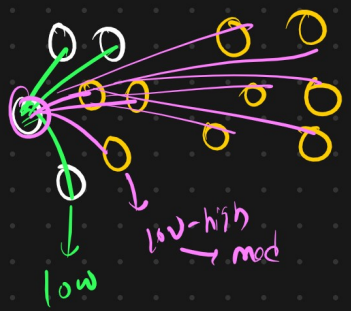


avg dist → low

avg dist → high

high + low → high

high



low-high → med

low

med ⟺ low

b − a → med

Calinski – Harabasz Index (variance Ratio Criterion)

Also called as CH or VRC evaluation

→ Measures ratio of :

- between-cluster variance
- within cluster variance

$$CH = \frac{(\text{between-cluster dispersion}/(k-1))}{(\text{within-cluster dispersion}/(n-k))} \longrightarrow \uparrow \text{higher is between}$$

→ best with k-means

n = number of samples

k = number of cluster

C1 → A (1,1)
→ B (2,1)

C2 → C (8,8)
D (9,8)

$k = 2$
$n = 2$

Step 1: Compute cluster mean

C1 mean: $\mu_1 = (1.5, 1)$

C2 mean: $\mu_2 = (8.5, 8)$

Global mean: $\mu = (5, 4.5)$

Step 2: Between-cluster dispersion

$$B = \sum_{i=1}^{k} n_i \cdot \|\mu_i - \mu\|^2$$

C1: $\|\mu_i - \mu\|^2 \rightarrow (1.5-5)^2 + (1-4.5)^2 = 24.5$

C2: $\|\mu_i - \mu\|^2 \rightarrow (8.5-5)^2 + (8-4.5)^2 = 24.5$

$$B = 2 \cdot 24.5 + 2 \cdot 24.5 = 98$$

Step 3: Within-cluster dispersion

Cluster 1: (A, B)

A: $(1-1.5)^2 + (1-1)^2 = 0.25$

B: $(2-1.5)^2 + (1-1)^2 = 0.25$

C1: 0.5

Cluster 2 : (10)

C : $(8-8.5)^2 + (8-8)^2 = 0.25$

D : $(9-8.5)^2 + (8-8)^2 = 0.25$

C2 : 0.5

Total within-cluster : C1 + C2

: 1.0

Step 4 : CH score : $\frac{(B/k-1)}{(W/n-k)} \rightarrow \frac{(98/1)}{(1/2)} = 98 \times 2 = 196$

high score means → cluster data is compact & cluster are far apart.

Interpretation :

0-20 → weak, cluster overlap, not separated

20-50 → Moderate clustering

50-150 → Good clustering

150+ → strong, well separated cluster

500+ → perfect seperation

```
X = load_data()
kmeans = KMean( n_clusters = X , random_state = 20 )
labels = kmeans.fit_predict (X)
from sklearn.metrics import silhouette_score, calinski_harabasz_score
Sil_score = silhouette_score( X, labels)
Ch_score = calinski_harabasz_score ( X, labels)
```

End of Machine Learning

Machine Learning project

User personal + financial record → approve → how much loan amount we can give them.
↓ → reject
100k $

ML Engineer:

→ Data Gathering → collect the data from the source.

→ Experimentation stage

→ .py → .ipynb

↓

→ Basic start) (EDA-I)

→ EDA-II

→ FE

→ Model selection

→ part-1 (classification)	→ part-2 (regression)
→ procesi the data for classification	→ procesi the data for regression
→ iterate over multiple models	→ iterate over multiple models
→ use classification metrics to find best mode.	→ use regression metrics to find best model.
→ once the model is selected, use hyper parameter tuning to find optimal parameters.	→ once the model is selected, use hyper parameter tuning to find optimal parameters.
→ save model & artefacts.	→ save model & artefacts.
→ create predict function to test the flow.	→ create predict function to test the flow.

→ Creation of runnable script (clean implementation of the code) (main.py)

- → load the model & artefact
- → predict function / method
- → OOP (optional)
- → Functionality to start from input & get the user desired output.

→

requirement.txt of the above solution.

main.py

models & artefacts

sent to a EC2 instance

deployment

api

↓

fastapi / flask

↓

api

↓

port:8080/predict/{data} → rejected
→ loan amount

